

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



HCMUTE

ĐỒ ÁN 1

THIẾT KẾ NHÀ THÔNG MINH TÍCH HỢP ĐIỀU KHIỂN GIỌNG NÓI

NGÀNH KỸ THUẬT MÁY TÍNH

GVHD: TS. Huỳnh Thế Thiện

STT	HỌ VÀ TÊN SV	MSSV
1	Phạm Phúc Gia Hiền	22119069
2	Nguyễn Anh Minh	22119103

TP. Hồ Chí Minh – 5/2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



HCMUTE

ĐỒ ÁN 1

THIẾT KẾ NHÀ THÔNG MINH
TÍCH HỢP ĐIỀU KHIỂN GIỌNG NÓI

NGÀNH KỸ THUẬT MÁY TÍNH

GVHD: TS. Huỳnh Thế Thiện

STT	HỌ VÀ TÊN SV	MSSV
1	Phạm Phúc Gia Hiền	22119069
2	Nguyễn Anh Minh	22119103

TP. Hồ Chí Minh – 5/2025

Lời cảm ơn

Trong quá trình thực hiện Đồ án môn học 1 với đề tài “Thiết kế hệ thống nhà thông minh tích hợp điều khiển giọng nói”, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn tận tình từ Thầy Huỳnh Thế Thiện.

Thầy không chỉ là người hướng dẫn chuyên môn mà còn là người truyền cảm hứng, giúp chúng em tiếp cận vấn đề một cách khoa học và thực tiễn. Chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến Thầy.

Nhóm thực hiện đồ án môn học 1
(Ký tên và ghi rõ họ tên)

Lời cam đoan

Nhóm thực hiện đồ án môn học 1 xin cam kết rằng toàn bộ nội dung trong đề tài được xây dựng dựa trên quá trình nghiên cứu, tìm hiểu tài liệu và thực hiện thực tế. Nhóm không sao chép nội dung hay kết quả từ bất kỳ đồ án nào khác. Các tài liệu tham khảo đã được trích dẫn rõ ràng và đầy đủ trong báo cáo.

Nhóm thực hiện đồ án môn học 1
(Ký tên và ghi rõ họ tên)

Tóm tắt

Hệ thống nhà thông minh được thiết kế nhằm tự động hóa quá trình giám sát và điều khiển các thiết bị trong gia đình, từ đó nâng cao sự tiện nghi, đảm bảo an toàn không gian sống. Hệ thống sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm, tích hợp với các thành phần phần cứng như cảm biến, keypad, LCD và các relay điều khiển thiết bị điện.

Các chức năng chính của hệ thống được triển khai dưới dạng các task độc lập sử dụng FreeRTOS, bao gồm:

- Giám sát nhiệt độ và độ ẩm: Thu thập dữ liệu từ cảm biến, hiển thị thông tin lên màn hình LCD và gửi dữ liệu theo thời gian thực lên ứng dụng Blynk.
- Điều khiển thiết bị qua keypad: Nhận tín hiệu từ bàn phím số để thực hiện các hành động như mở khóa bằng servo.
- Hệ thống chiếu sáng thông minh: Sử dụng cảm biến chuyển động để phát hiện khi có người, đèn sẽ tự động bật nhằm tăng tiện nghi và tiết kiệm năng lượng.
- Giám sát an toàn phòng bếp: Phát hiện khí gas và cảnh báo người dùng thông qua còi cảnh báo khi có dấu hiệu rò rỉ khí.
- Điều khiển phòng ngủ: Cho phép người dùng điều khiển quạt và đèn bằng nút nhấn ảo thông qua Blynk.

Ngoài ra, hệ thống còn hỗ trợ tương tác từ xa thông qua mạng Internet và được tích hợp điều khiển bằng giọng nói thông qua trợ lý ảo Siri, góp phần nâng cao tính linh hoạt, hiện đại cho hệ thống.

Mục lục

1	TỔNG QUAN	6
1.1	Giới thiệu	6
1.2	Mục tiêu của đề tài	6
1.3	Phương pháp nghiên cứu	6
1.4	Giới hạn đề tài	7
1.5	Bố cục nội dung	8
2	CƠ SỞ LÝ THUYẾT	9
2.1	Hướng xây dựng hệ thống	9
2.2	Hệ thống nhúng và vi điều khiển	10
2.2.1	Khái niệm hệ thống nhúng	10
2.2.2	Vi điều khiển	10
2.2.3	Hệ điều hành thời gian thực RTOS	11
2.2.4	Quản lý bộ nhớ trong FreeRTOS	13
2.3	Internet of Things (IoT)	14
2.3.1	Giao thức truyền thông IoT	14
2.3.2	Tầng giao vận trong kiến trúc IoT	14
2.3.3	Kiến trúc client-server trong IoT	15
2.3.4	Blynk	15
2.3.5	Siri	16
2.4	Giao tiếp và kết nối trong hệ thống	16
2.5	Vi điều khiển và kết nối mạng không dây	17
3	THIẾT KẾ	18
3.1	Yêu cầu người dùng	18
3.2	Yêu cầu kỹ thuật của hệ thống	18
3.2.1	Yêu cầu chức năng	18

3.2.2	Yêu cầu phi chức năng	19
3.3	Kiến trúc hệ thống	20
3.4	Thiết kế phần cứng	21
3.5	Thiết kế phần mềm	26
3.5.1	Thiết lập Blynk	26
3.5.2	Thiết lập Siri và Blynk	27
4	THI CÔNG VÀ ĐÁNH GIÁ	29
4.1	Thi công	29
4.1.1	Thi công phần cứng	29
4.1.2	Nguyên lí hoạt động	30
4.1.3	Lưu đồ giải thuật	31
4.2	Kết quả	34
4.3	Đánh giá hệ thống	39
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	40
5.1	Kết luận	40
5.2	Hướng phát triển	41
	TÀI LIỆU THAM KHẢO	42

Danh sách hình vẽ

2.1.1 Mô hình thực hiện chương trình	9
2.2.1 Vi điều khiển ESP32	10
2.2.2 FreeRTOS Flow Chart	11
2.2.3 Sơ đồ trạng thái tác vụ trong FreeRTOS	12
2.3.1 Giao tiếp Client - Server	15
3.3.1 Sơ đồ khối hệ thống	20
3.4.1 ESP32 WROOM 32.	22
3.4.2 Hình 3.5.2: Sơ đồ chân ESP32 WROOM 32.	22
3.4.3 Cảm biến nhiệt độ, độ ẩm DHT11	23
3.4.4 Cảm biến PIR HC - SR501	24
3.4.5 Màn hình LCD 16x02	24
3.4.6 Động cơ servo SG90	25
3.5.1 Màn hình quản lý của Blynk	26
3.5.2 Tạo devices cho Blynk	26
3.5.3 Tạo công tắc ảo	27
3.5.4 Màn hình Blynk sau khi tạo các công tắc ảo	27
3.5.5 Dữ liệu từ Siri đến Devices	27
4.1.1 Sơ đồ kết nối các module phần cứng và phần mềm.	29
4.1.2 Sơ đồ kết nối.	29
4.1.3 Sơ đồ nguyên lí.	30
4.1.4 Lưu đồ tổng quát của hệ thống nhà thông minh.	31
4.1.5 Lưu đồ phòng khách của hệ thống nhà thông minh.	31
4.1.6 Lưu đồ phòng bếp của hệ thống nhà thông minh.	32
4.1.7 Lưu đồ phòng ngủ của hệ thống nhà thông minh.	33
4.2.1 Mô hình sản phẩm	34
4.2.2 Cửa mở khi mật khẩu đúng.	35

4.2.3 Cửa mở khi mật khẩu đúng.	35
4.2.4 Giao diện blynk hiển thị nhiệt độ, độ ẩm.	36
4.2.5 Giao diện Blynk thông báo nguy hiểm.	36
4.2.6 Đèn và còi sẽ hú khi nguy hiểm,	37
4.2.7 Sử dụng Siri thành công	37
4.2.8 Đèn đã sáng và quạt đã xoay.	38
4.2.9 Giao diện blynk mà người dùng sử dụng Siri bật đèn và quạt	38

Danh sách bảng

3.3.1 Mô tả chức năng các khối	20
3.4.1 Mô tả chức năng các khối	21

Chương 1

TỔNG QUAN

1.1 Giới thiệu

Trong bối cảnh cuộc sống hiện đại ngày càng đề cao sự tiện nghi, an toàn và tiết kiệm năng lượng, việc ứng dụng các công nghệ thông minh vào không gian sống dần trở thành xu hướng tất yếu. Đặc biệt, trong các hộ gia đình, nhu cầu tự động hóa các thiết bị điện, giám sát môi trường sống và điều khiển từ xa thay thế các phương pháp tương tác cơ học ngày càng gia tăng, điều này đòi hỏi cần có một giải pháp tích hợp đa chức năng, có khả năng vận hành linh hoạt và hỗ trợ điều khiển thông minh, bao gồm cả điều khiển bằng giọng nói.

Nhờ vào sự phát triển mạnh mẽ của các nền tảng IoT và sự ra đời của các vi điều khiển với khả năng xử lý mạnh mẽ, việc triển khai một hệ thống nhà thông minh trở nên khả thi và hiệu quả hơn bao giờ hết. Từ những lý do trên, nhóm chúng em quyết định nghiên cứu và phát triển đề tài này, với mong muốn đóng góp một giải pháp công nghệ hữu ích, thực tiễn trong đời sống.

1.2 Mục tiêu của đề tài

Mục tiêu của đề tài là xây dựng một hệ thống nhà thông minh có khả năng giám sát và điều khiển các thiết bị, cho phép người dùng tương tác thông qua nhiều hình thức khác nhau, bao gồm điều khiển từ xa bằng ứng dụng và điều khiển bằng giọng nói thông qua trợ lý ảo. Bên cạnh đó, hệ thống còn hỗ trợ theo dõi môi trường sống, nâng cao mức độ an toàn trong sinh hoạt. Đề tài hướng đến khả năng ứng dụng thực tế trong đời sống, góp phần cải thiện chất lượng cuộc sống và thúc đẩy xu hướng tự động hóa không gian sống hiện đại.

1.3 Phương pháp nghiên cứu

- **Nghiên cứu tài liệu:** Tìm hiểu các công nghệ liên quan đến nhà thông minh, phương pháp điều khiển thiết bị từ xa qua mạng, nguyên lý hoạt động của các cảm biến và cách kết nối với các nền tảng hỗ trợ điều khiển bằng giọng nói.

- **Phát triển mô hình:** Nhóm tiến hành xây dựng mô hình hệ thống dựa trên một nền tảng phần cứng nhỏ gọn, có khả năng kết nối không dây và điều khiển nhiều chức năng cùng lúc. Các chức năng được tổ chức thành từng tiến trình riêng biệt, hoạt động song song và phối hợp thông qua một bộ xử lý trung tâm có khả năng vận hành đa nhiệm trong thời gian thực, giúp tăng độ ổn định và khả năng phản hồi của hệ thống.
- **Phân tích và đánh giá:**
 - Đánh giá hiệu quả hoạt động: Kiểm tra khả năng phản hồi, độ chính xác và độ ổn định của hệ thống trong các tình huống như mất kết nối mạng, thay đổi môi trường hoặc hoạt động liên tục trong thời gian dài.
 - Thử nghiệm thực tế: Hệ thống được triển khai thử nghiệm trong mô hình phần cứng để đánh giá hiệu năng khi điều khiển qua Blynk và phản hồi giọng nói thông qua trợ lý ảo Siri.
 - Phân tích hạn chế: Nhận diện các yếu tố có thể ảnh hưởng đến hiệu suất hệ thống như tốc độ kết nối mạng, điều kiện môi trường, khả năng nhận dạng chính xác các lệnh thoại hoặc giới hạn về tài nguyên của phần cứng.
 - Đề xuất cải tiến: Dựa trên các kết quả thử nghiệm, nhóm đề xuất các giải pháp nâng cao hiệu quả điều khiển, cải thiện độ ổn định và mở rộng thêm các chức năng trong tương lai.

1.4 Giới hạn đề tài

Đề tài này tập trung xây dựng mô hình thử nghiệm cho hệ thống nhà thông minh, hỗ trợ tích hợp điều khiển giọng nói thông qua trợ lý ảo Tuy nhiên, vẫn còn một số giới hạn nhất định trong quá trình thực hiện, bao gồm:

- Hệ thống chỉ dừng ở mức mô hình thử nghiệm, chưa đủ điều kiện để triển khai thực tế trong môi trường nhà ở hoàn chỉnh.
- Các chức năng được xây dựng dưới dạng cơ bản, chưa tích hợp các thuật toán phân tích dữ liệu nâng cao hay cơ chế học thông minh để tự điều chỉnh hành vi hệ thống theo thói quen người dùng.
- Phụ thuộc vào kết nối Internet để gửi dữ liệu cảm biến và nhận lệnh điều khiển từ xa; khi mất mạng, hệ thống có thể ngừng hoạt động hoặc phản hồi chậm.
- Việc tích hợp điều khiển bằng giọng nói thông qua Siri phụ thuộc vào thiết bị trung gian và nền tảng hỗ trợ, nên còn giới hạn về khả năng tương thích và tính linh hoạt trong các môi trường hệ điều hành khác nhau.

1.5 Bố cục nội dung

Báo cáo đồ án môn học 1 của nhóm bao gồm 5 chương:

- Chương 1: Tổng quan.
- Chương 2: Cơ sở lý thuyết.
- Chương 3: Thiết kế hệ thống.
- Chương 4: Thi công và đánh giá.
- Chương 5: Kết luận và hướng phát triển.

Chương 2

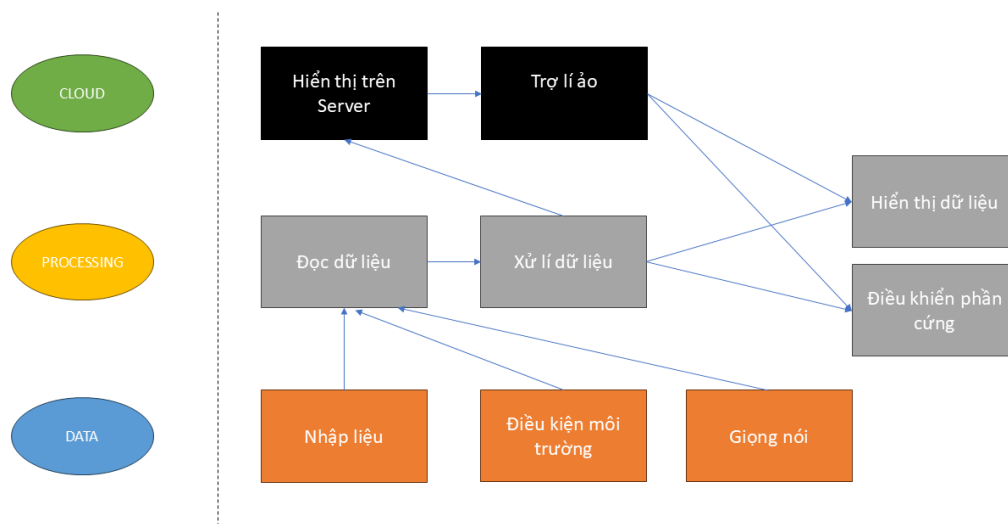
CƠ SỞ LÝ THUYẾT

2.1 Hướng xây dựng hệ thống

Hệ thống nhà thông minh được chia thành hai phần chính là phần cứng và phần mềm. Phần cứng bao gồm các cảm biến, thiết bị điều khiển và các module hiển thị hoặc nhập liệu. Phần mềm được xây dựng dựa trên nền tảng Internet of Things (IOT) để đảm bảo khả năng giao tiếp linh hoạt giữa người dùng và hệ thống.

Trong quá trình thiết kế hệ thống, chúng em tiến hành so sánh và lựa chọn các thiết bị và giao thức phù hợp dựa trên các yếu tố như: tính ổn định khi kết nối mạng không dây, khả năng xử lý đồng thời nhiều tác vụ và độ dễ dàng khi tích hợp với các nền tảng.

Sau khi lựa chọn phần cứng và nền tảng điều khiển phù hợp, chương trình điều khiển được viết bằng Arduino IDE và triển khai trên vi điều khiển. Giao diện người dùng được thiết kế thông qua ứng dụng Blynk, giúp người dùng dễ dàng điều khiển các thiết bị và giọng nói.



Hình 2.1.1: Mô hình thực hiện chương trình

2.2 Hệ thống nhúng và vi điều khiển

Trong bối cảnh thiết bị thông minh ngày càng trở nên phổ biến và tích hợp sâu vào đời sống, hệ thống nhúng và vi điều khiển đã nổi lên như một nền tảng cốt lõi, cho phép hiện thực hóa các chức năng điều khiển, giám sát và tự động hóa một cách linh hoạt và hiệu quả. Thay vì sử dụng các máy tính truyền thống với cấu trúc phức tạp và tiêu tốn tài nguyên, hệ thống nhúng mang đến giải pháp nhỏ gọn, tiết kiệm năng lượng nhưng vẫn đảm bảo khả năng phản hồi nhanh và đáng tin cậy.

2.2.1 Khái niệm hệ thống nhúng

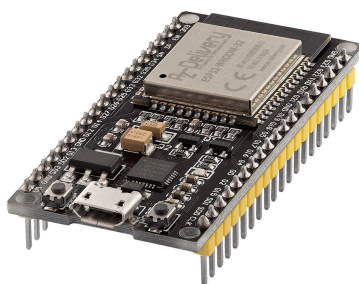
Hệ thống nhúng (Embedded System) là một thuật ngữ để chỉ một hệ thống có khả năng tự trị được nhúng vào trong một môi trường hay một hệ thống mẹ. Đó là các hệ thống tích hợp cả phần cứng và phần mềm phục vụ các bài toán chuyên dụng trong nhiều lĩnh vực công nghiệp, tự động hoá điều khiển, quan trắc và truyền tin. Đặc điểm của các hệ thống nhúng là hoạt động ổn định và có tính năng tự động hoá cao.

Hệ thống nhúng thường được thiết kế để thực hiện một chức năng chuyên biệt nào đó. Một hệ thống nhúng chỉ thực hiện một hoặc một vài chức năng nhất định, thường đi kèm với những yêu cầu cụ thể và bao gồm một số thiết bị máy móc và phần cứng chuyên dụng mà ta không tìm thấy trong một máy tính đa năng nói chung.

Trong hệ thống điều khiển và giám sát tự động, hệ thống nhúng mang lại khả năng tích hợp cảm biến, giao tiếp không dây và điều khiển thiết bị, từ đó nâng cao tính linh hoạt và hiệu quả trong các ứng dụng như tự động hóa nhà ở, an toàn môi trường và tương tác người-máy.

2.2.2 Vi điều khiển

Vi điều khiển (Microcontroller) là một loại mạch tích hợp (IC) được thiết kế để thực hiện các tác vụ điều khiển trong các hệ thống nhúng. Về cơ bản, một vi điều khiển tích hợp đầy đủ các thành phần chức năng cần thiết của một hệ thống máy tính thu nhỏ trên một chip duy nhất, bao gồm: bộ xử lý trung tâm (CPU), bộ nhớ (RAM, ROM hoặc Flash), các thiết bị ngoại vi (như bộ định thời – timer, bộ chuyển đổi tương tự – số, giao tiếp nối tiếp như UART, SPI, I2C), và các cổng vào/ra (GPIO).



Hình 2.2.1: Vi điều khiển ESP32

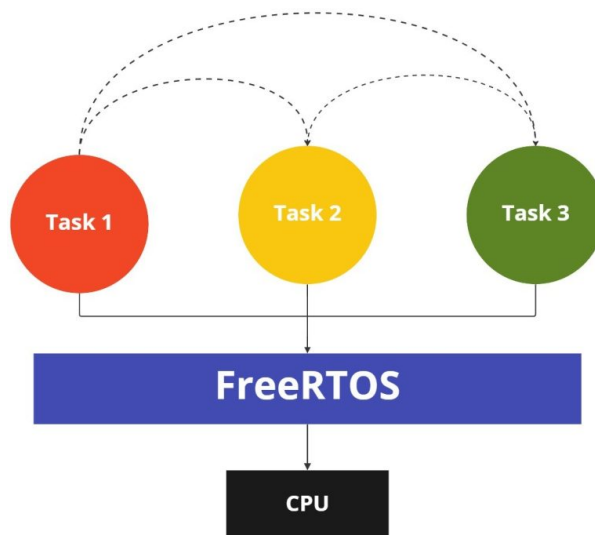
Cấu trúc này cho phép vi điều khiển thực hiện các tác vụ điều khiển theo thời gian thực, xử lý tín hiệu từ cảm biến, điều khiển cơ cấu chấp hành, và giao tiếp với các thiết bị khác một cách hiệu quả. Vi điều khiển thường được tối ưu hóa cho các ứng dụng cụ thể, có kích thước nhỏ gọn, tiêu thụ năng lượng thấp và khả năng hoạt động ổn định trong nhiều điều kiện môi trường. Nhờ đó, chúng được ứng dụng rộng rãi trong các thiết bị điện tử tiêu dùng, hệ thống tự động hóa, thiết bị y tế, và đặc biệt là trong các hệ thống điều khiển nhúng thông minh.

2.2.3 Hệ điều hành thời gian thực RTOS

Hệ điều hành thời gian thực (Real-Time Operating System - RTOS) là một loại hệ điều hành được thiết kế để xử lý các tác vụ trong khoảng thời gian xác định, đảm bảo tính chính xác và độ tin cậy trong các hệ thống nhúng. RTOS thường được sử dụng trong các ứng dụng yêu cầu đáp ứng nhanh và ổn định như thiết bị IoT, điều khiển công nghiệp, robot và hệ thống điều khiển tự động.

FreeRTOS

FreeRTOS là một hệ điều hành thời gian thực mã nguồn mở, nhẹ và phổ biến, được thiết kế để chạy trên các vi điều khiển có tài nguyên hạn chế. FreeRTOS hỗ trợ đa nhiệm với khả năng lập lịch theo ưu tiên, cho phép nhiều tác vụ chạy song song và chia sẻ CPU hiệu quả. Với cộng đồng phát triển mạnh mẽ và tài liệu hỗ trợ phong phú, FreeRTOS được sử dụng rộng rãi trong các hệ thống nhúng, đặc biệt là trong các ứng dụng IoT, điều khiển thiết bị và tự động hóa.

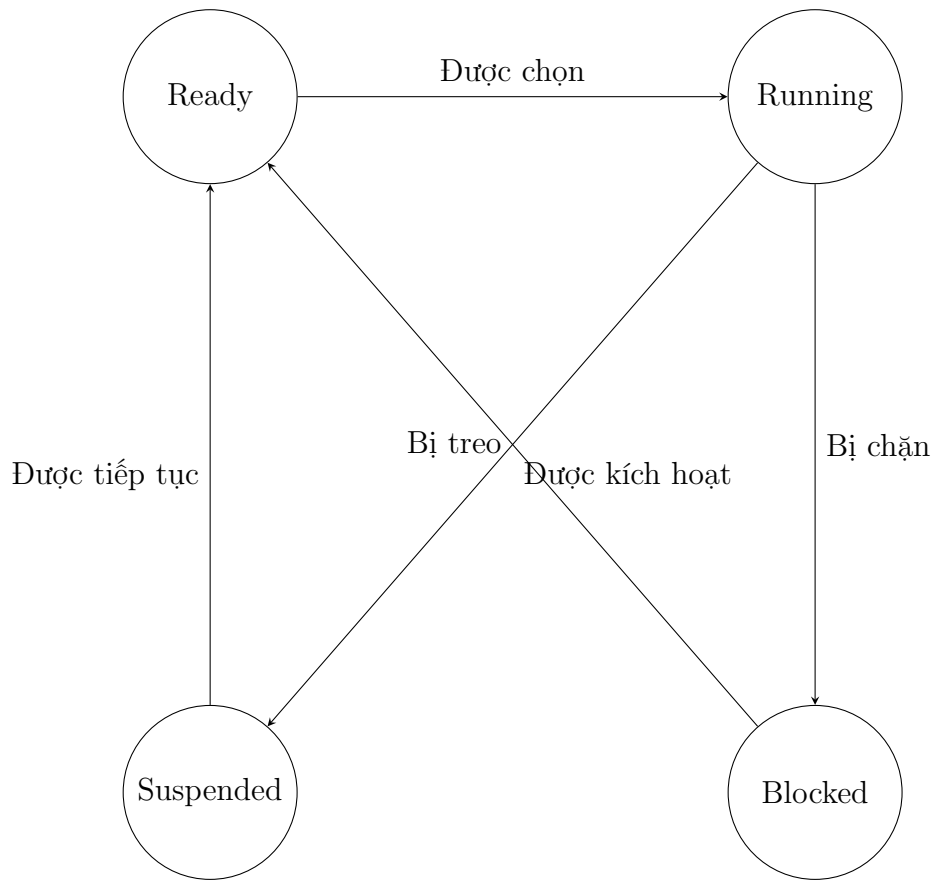


Hình 2.2.2: FreeRTOS Flow Chart

Quản lý tác vụ

Trong FreeRTOS, mỗi tác vụ (task) được coi như một luồng thực thi độc lập với hàm xử lý riêng, ngăn xếp riêng và trạng thái riêng. Hệ điều hành sử dụng bộ lập lịch để quản

lý và phân phối thời gian CPU giữa các tác vụ dựa trên độ ưu tiên. Các tác vụ có thể được tạo, tạm dừng, tiếp tục hoặc xoá bằng cách sử dụng các hàm API. Việc quản lý tác vụ hiệu quả giúp hệ thống hoạt động ổn định, đáp ứng thời gian thực và tối ưu hóa tài nguyên xử lý.



Hình 2.2.3: Sơ đồ trạng thái tác vụ trong FreeRTOS

Cơ chế đồng bộ (Task Synchronization)

Trong FreeRTOS, đồng bộ giữa các tác vụ hoặc giữa tác vụ và ngắt là rất quan trọng để đảm bảo tính nhất quán dữ liệu và điều phối hoạt động hệ thống. FreeRTOS cung cấp nhiều cơ chế đồng bộ linh hoạt và hiệu quả, phù hợp cho các hệ thống nhúng thời gian thực.

- Semaphore: Là công cụ phổ biến dùng để đồng bộ giữa các tác vụ. Gồm có:
 - Binary Semaphore: Đồng bộ đơn giản giữa hai đối tượng.
 - Counting Semaphore: Hữu ích khi cần đếm số lượng tài nguyên hoặc sự kiện.
- Queues (Hàng đợi): Là cấu trúc dữ liệu cơ bản để truyền dữ liệu giữa các tác vụ hoặc giữa tác vụ và ngắt. Dữ liệu được lưu trữ theo thứ tự FIFO (First In, First Out).
- Mutex (Mutual Exclusion): Được sử dụng để bảo vệ tài nguyên dùng chung, đảm bảo chỉ một tác vụ được truy cập tại một thời điểm. FreeRTOS hỗ trợ cả priority inheritance (Kế thừa ưu tiên) để tránh tình trạng ưu tiên bị đảo.

- Task Notifications: là một cơ chế truyền thông và đồng bộ nhẹ, được tích hợp sẵn trong mỗi tác vụ của FreeRTOS. Mỗi tác vụ có một biến thông báo 32-bit dùng để nhận dữ liệu hoặc tín hiệu từ các tác vụ khác hoặc từ ngắt (ISR), với chi phí thấp hơn so với việc sử dụng semaphore hay queue.
- Event Groups: Cho phép đồng bộ nhiều tác vụ dựa trên các cờ sự kiện (event flags), phù hợp cho tình huống cần chờ nhiều điều kiện đồng thời.

Các cơ chế này giúp đảm bảo rằng các tác vụ có thể phối hợp hoạt động hiệu quả, giảm thiểu xung đột và tăng tính ổn định của hệ thống nhúng thời gian thực.

2.2.4 Quản lý bộ nhớ trong FreeRTOS

FreeRTOS cung cấp cơ chế quản lý bộ nhớ động linh hoạt để phục vụ việc tạo các tác vụ (tasks), hàng đợi (queues), bộ đệm (buffers) và các đối tượng đồng bộ hóa như semaphore hoặc mutex. Việc quản lý bộ nhớ đúng cách là yếu tố then chốt giúp hệ thống nhúng hoạt động ổn định, tránh rò rỉ bộ nhớ hoặc lỗi tràn ngăn xếp.

FreeRTOS hỗ trợ nhiều trình quản lý bộ nhớ (heap memory allocation schemes) khác nhau, từ heap_1.c đến heap_5.c, cho phép lập trình viên lựa chọn cơ chế phù hợp với tài nguyên phần cứng:

- heap_1: đơn giản, không hỗ trợ giải phóng bộ nhớ (phù hợp cho ứng dụng nhỏ, tĩnh).
- heap_2: có thể cấp phát và giải phóng, sử dụng danh sách liên kết đơn.
- heap_3: sử dụng malloc/free chuẩn của C (phụ thuộc vào bộ quản lý của hệ thống).
- heap_4: cải tiến từ heap_2, giảm phân mảnh bộ nhớ.
- heap_5: hỗ trợ quản lý nhiều vùng bộ nhớ (useful for large or fragmented memory layouts).

FreeRTOS cũng hỗ trợ hook function để xử lý khi thiếu bộ nhớ, như vApplicationMallocFailedHook(), giúp phát hiện và xử lý lỗi bộ nhớ kịp thời.

Xử lý ngắt

Xử lý ngắt trong FreeRTOS cho phép hệ thống nhúng phản hồi tức thời các sự kiện phần cứng, như tín hiệu từ cảm biến hoặc lệnh điều khiển, mà không cần thăm dò liên tục. Khi một ngắt xảy ra, FreeRTOS tạm dừng tác vụ hiện tại, thực thi hàm xử lý ngắt, và nhanh chóng quay lại tác vụ, đảm bảo hiệu suất thời gian thực. Các cơ chế như hàng đợi hoặc semaphore được sử dụng để truyền dữ liệu từ ngắt đến tác vụ, tránh xung đột.

Lập lịch và phân phối tác vụ

FreeRTOS sử dụng cơ chế lập lịch dựa trên ưu tiên (priority-based scheduling), ưu tiên các tác vụ quan trọng để đáp ứng yêu cầu thời gian thực. Mỗi tác vụ được phân phối tài nguyên theo mức ưu tiên, giúp tối ưu hiệu suất trong điều kiện tài nguyên hạn chế. Trong hệ thống IoT, lập lịch và phân phối tác vụ cho phép vi điều khiển quản lý trơn tru các hoạt động như giám sát môi trường hoặc phản hồi lệnh từ xa, đảm bảo độ chính xác và tin cậy cho các ứng dụng tự động hóa thông minh.

2.3 Internet of Things (IoT)

IoT (Internet of Things) hay Vạn vật kết nối Internet là hệ thống kết nối các thiết bị vật lý và thiết bị ảo qua Internet, mỗi thiết bị đều được cung cấp một định danh và có khả năng thu thập, trao đổi và xử lý dữ liệu thông qua các cảm biến và thiết bị giám sát mà không yêu cầu tương tác trực tiếp giữa máy tính và con người.

2.3.1 Giao thức truyền thông IoT

Giao thức truyền thông IoT đảm bảo kết nối và truyền dữ liệu giữa các thiết bị. Các giao thức chính gồm:

- Message Queuing Telemetry Transport (MQTT): Giao thức nhẹ, sử dụng mô hình publish/subscribe, phù hợp cho các thiết bị hạn chế tài nguyên.
- Constrained Application Protocol (CoAP): Được thiết kế cho môi trường băng thông thấp, hỗ trợ giao tiếp RESTful.
- HyperText Transfer Protocol/Secure (HTTP/HTTPS): Phổ biến trong các ứng dụng web, nhưng tiêu tốn nhiều tài nguyên trong IoT.
- Long Range Wide Area Network (LoRaWAN): Phù hợp cho mạng diện rộng, tiết kiệm năng lượng, hỗ trợ truyền dữ liệu khoảng cách xa.

Việc lựa chọn giao thức phụ thuộc vào yêu cầu về năng lượng, băng thông và độ trễ của hệ thống IoT.

2.3.2 Tầng giao vận trong kiến trúc IoT

Trong kiến trúc mạng IoT, bên cạnh các giao thức ứng dụng như MQTT, CoAP hay HTTP, các giao thức tầng giao vận đóng vai trò nền tảng đảm bảo việc truyền tải dữ liệu giữa các thiết bị đầu cuối và máy chủ. Hai giao thức phổ biến nhất ở tầng này là:

- TCP (Transmission Control Protocol): Là giao thức hướng kết nối, cung cấp khả năng kiểm soát lỗi và đảm bảo dữ liệu được truyền đến đích đầy đủ và theo đúng thứ tự. TCP thường được sử dụng trong các giao thức IoT như MQTT và HTTP, nơi cần độ tin cậy cao trong việc truyền dữ liệu, chẳng hạn như điều khiển thiết bị hoặc thu thập thông tin cảm biến.

- UDP (User Datagram Protocol): Là giao thức không hướng kết nối, truyền dữ liệu với độ trễ thấp nhưng không đảm bảo độ tin cậy. UDP thường được sử dụng trong các ứng dụng yêu cầu thời gian thực hoặc trong các giao thức nhẹ như CoAP, nơi việc mất gói nhỏ lẻ có thể chấp nhận được để đổi lấy tốc độ và hiệu năng cao hơn.

Việc lựa chọn giữa TCP và UDP phụ thuộc vào đặc điểm và yêu cầu cụ thể của ứng dụng IoT. Những hệ thống cần độ tin cậy cao như giám sát an ninh hoặc điều khiển thiết bị chính xác thường ưu tiên TCP. Trong khi đó, các ứng dụng cảm biến đơn giản hoặc truyền tải định kỳ với khối lượng dữ liệu nhỏ và yêu cầu độ trễ thấp có thể tận dụng lợi thế của UDP.

2.3.3 Kiến trúc client-server trong IoT

Kiến trúc client-server trong IoT là mô hình giao tiếp phổ biến, trong đó thiết bị IoT (client) gửi yêu cầu đến máy chủ (server) để xử lý và nhận phản hồi.



Hình 2.3.1: Giao tiếp Client - Server

- Client: Các thiết bị IoT như cảm biến, thiết bị thông minh, thu thập dữ liệu và gửi yêu cầu đến server.
- Server: Hệ thống trung tâm xử lý dữ liệu, lưu trữ và trả về kết quả cho client.
- Giao tiếp: Thường sử dụng các giao thức như HTTP/HTTPS hoặc MQTT để đảm bảo truyền dữ liệu hiệu quả.

Kiến trúc này phù hợp cho các ứng dụng IoT yêu cầu xử lý tập trung, nhưng có thể gặp thách thức về độ trễ và phụ thuộc vào máy chủ.

2.3.4 Blynk

Blynk là một nền tảng IoT cho phép kết nối thiết bị phần cứng với ứng dụng di động và đám mây để điều khiển và giám sát từ xa. Nền tảng này cung cấp các thành phần chính:

- Blynk App: Ứng dụng di động trên iOS và Android, hỗ trợ tạo giao diện người dùng bằng cách kéo thả các widget như nút, thanh trượt, biểu đồ để điều khiển và hiển thị dữ liệu.

- Blynk Cloud: Hệ thống máy chủ đám mây đảm bảo giao tiếp an toàn, lưu trữ dữ liệu thời gian thực và quản lý thiết bị.
- Blynk Library: Thư viện mã nguồn mở hỗ trợ kết nối hơn 400 loại phần cứng như Arduino, ESP8266, ESP32, Raspberry Pi qua các giao thức như WiFi, Ethernet, MQTT.

Blynk phù hợp cho cả ứng dụng cá nhân và thương mại, cho phép phát triển nhanh các dự án IoT mà không cần viết nhiều mã, đồng thời đảm bảo tính bảo mật và khả năng mở rộng.

2.3.5 Siri

Siri là trợ lý ảo do Apple phát triển, được tích hợp sẵn trên các thiết bị của hãng như iPhone, iPad, Apple Watch,... Trong hệ thống nhà thông minh, Siri đóng vai trò trung gian giữa người dùng và các thiết bị IoT, cho phép điều khiển thiết bị bằng giọng nói một cách tự nhiên và tiện lợi.

Về mặt kiến trúc, Siri hoạt động dựa trên mô hình client-server. Khi người dùng ra lệnh bằng giọng nói, dữ liệu âm thanh sẽ được mã hóa và gửi đến máy chủ của Apple thông qua giao thức HTTP/HTTPS để xử lý ngôn ngữ tự nhiên (NLP - Natural Language Processing). Sau khi xử lý xong, máy chủ sẽ gửi phản hồi trở lại thiết bị để thực thi hành động.

Để điều khiển thiết bị IoT, Siri thường được tích hợp với nền tảng HomeKit của Apple. HomeKit đóng vai trò là một hub điều phối, quản lý và giao tiếp với các thiết bị thông minh trong hệ sinh thái Apple. Tuy nhiên, trong một số giải pháp tùy chỉnh, người dùng có thể sử dụng Shortcuts (Phím tắt) để gọi các API bên ngoài, ví dụ như gửi HTTP Request đến server Blynk hoặc một ESP32 đang chạy web server, giúp mở rộng khả năng điều khiển giọng nói đến các thiết bị không nằm trong hệ sinh thái HomeKit.

2.4 Giao tiếp và kết nối trong hệ thống

Hệ thống được thiết kế để tích hợp nhiều thành phần cảm biến, thiết bị ngoại vi, tất cả được điều khiển bởi vi điều khiển ESP32. Các giao thức truyền thông và phương thức kết nối được sử dụng, bao gồm cả điều khiển từ xa qua Blynk và tích hợp giọng nói với Siri:

- Wi-Fi: Được sử dụng để kết nối ESP32 với mạng Internet, cho phép giao tiếp với máy chủ Blynk. Điều này hỗ trợ gửi dữ liệu cảm biến (nhiệt độ, độ ẩm, trạng thái khí gas) đến các chân ảo và nhận lệnh điều khiển từ ứng dụng Blynk.
- HTTP/HTTPS: Giao tiếp giữa ESP32 và máy chủ Blynk, cho phép gửi dữ liệu cảm biến và nhận các lệnh điều khiển từ giao diện ứng dụng Blynk hoặc qua Siri.
- I2C (Inter-Integrated Circuit): Được sử dụng để giao tiếp giữa ESP32 và màn hình LCD.
- Digital Input/Output (GPIO): Các chân GPIO của vi điều khiển được sử dụng để điều khiển các thiết bị ngoại vi.

- Điều khiển giọng nói qua Siri: Hệ thống tích hợp với Siri thông qua Shortcut và Blynk, cho phép người dùng sử dụng lệnh thoại để điều khiển các thiết bị.

2.5 Vi điều khiển và kết nối mạng không dây

Vi điều khiển ESP32 đóng vai trò là trung tâm điều khiển chính trong hệ thống nhà thông minh. Nó thực hiện việc thu thập dữ liệu từ các cảm biến và điều khiển các thiết bị đầu ra. Nhờ tích hợp sẵn Wi-Fi, ESP32 có khả năng kết nối mạng không dây, cho phép giao tiếp với nền tảng Blynk để điều khiển từ xa qua điện thoại hoặc bằng giọng nói qua Siri. Điều này mang lại sự tiện lợi và linh hoạt cho người dùng mà không cần dây kết nối vật lý.

Chương 3

THIẾT KẾ

3.1 Yêu cầu người dùng

- Độ chính xác và tin cậy: Hệ thống phải hoạt động ổn định và không bị gián đoạn, cung cấp kết quả tin cậy trong thời gian dài sử dụng.
- Thời gian phản hồi nhanh: Khi người dùng điều khiển các thiết bị thông qua Blynk, hệ thống phản hồi ngay lập tức.
- Hệ thống sử dụng giao diện Blynk để người dùng thực hiện các thao tác điều khiển thiết bị, xem trạng thái nhiệt độ, độ ẩm và nhận thông báo khi phát hiện nguy hiểm.
- Người dùng sử dụng mật khẩu để mở khóa cửa trước khi vào nhà.
- Người dùng có thể xem trạng thái nhiệt độ, độ ẩm thông qua màn hình LCD.
- Người dùng được cảnh báo thông qua LED, buzzer khi phát hiện rò rỉ khí gas.
- Người dùng sử dụng giọng nói để điều khiển các thiết bị trong nhà.
- Hệ thống nhỏ gọn, dễ bảo trì và sửa chữa khi có lỗi xảy ra.

3.2 Yêu cầu kĩ thuật của hệ thống

3.2.1 Yêu cầu chức năng

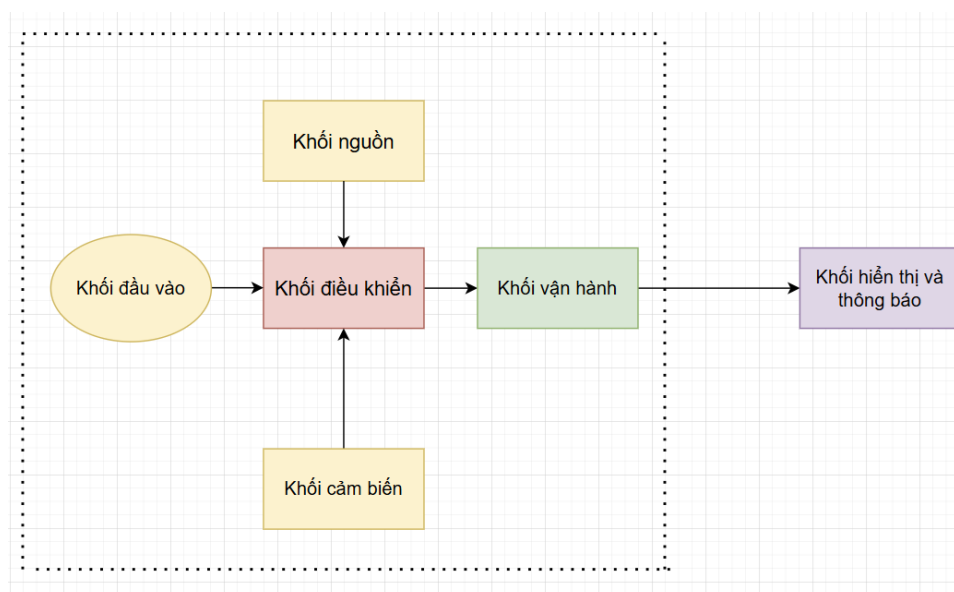
- **Bảo mật mở cửa:** Hệ thống được trang bị cơ chế bảo mật bằng mật khẩu để kiểm soát việc mở cửa, đảm bảo chỉ những người dùng biết được mật khẩu mới có thể vào nhà.
- **Giám sát và điều khiển thiết bị:** Cho phép người dùng điều khiển các thiết bị trong hệ thống thông qua Blynk hoặc bằng giọng nói thông qua điện thoại thông minh. Hệ thống ghi lại đầy đủ các thao tác bao gồm:
 - Tên thiết bị

- Hành động điều khiển (bật/tắt)
- **Phản hồi trạng thái thiết bị:** Hiển thị trạng thái hoạt động của thiết bị trên màn hình LCD hoặc trên Blynk bao gồm:
 - Tên thiết bị
 - Trạng thái hiện tại
 - Cảnh báo nguy hiểm

3.2.2 Yêu cầu phi chức năng

- **Hiệu suất:**
 - Thời gian phản hồi thao tác điều khiển thiết bị tối đa 3 giây.
 - Cập nhật trạng thái thiết bị trên giao diện người dùng trong vòng tối đa 1 giây sau khi thiết bị thay đổi trạng thái.
- **Bảo mật:** Chỉ người dùng đã đăng nhập và có quyền quản trị mới được phép cấu hình hoặc điều khiển các thiết bị.
- **Khả năng mở rộng:**
 - Hỗ trợ tích hợp thêm các cảm biến hoặc các khu vực mới trong nhà.
 - Hỗ trợ tích hợp công nghệ nhận diện khuôn mặt để tăng cường an ninh trong quá trình mở cửa.
- **Giao diện người dùng:**
 - Giao diện Blynk và ứng dụng trực quan, dễ sử dụng.
 - Hiển thị danh sách thiết bị, trạng thái hoạt động và thông số môi trường rõ ràng.

3.3 Kiến trúc hệ thống



Hình 3.3.1: Sơ đồ khối hệ thống

Khối	Mô tả
Khối cảm biến	- Chịu trách nhiệm thu thập dữ liệu đầu vào, thường là dữ liệu ở môi trường bên ngoài như là nhiệt độ, độ ẩm, chuyển động, phát hiện khí gas. - Cung cấp dữ liệu cho khối điều khiển để xử lý và ra quyết định. - Thiết bị này có thể là DTH11, PIR, MQ-2.
Khối điều khiển	- Nhận tín hiệu từ thiết bị đầu vào. - Xử lý logic, điều khiển các thiết bị đầu ra tương ứng. - Giao tiếp với nền tảng Blynk để gửi dữ liệu cảm biến và nhận lệnh điều khiển từ xa. - Thiết bị này là ESP32.
Khối đầu vào	- Cho phép người nhập mật khẩu mở cửa chính - Thiết bị này là bàn phím ma trận 4x3
Khối vận hành	- Thực thi các hành động cụ thể do hệ thống điều khiển đưa ra.
Khối nguồn	- Có nhiệm vụ cung cấp nguồn điện cho các thành phần trong hệ thống và đảm bảo hệ thống hoạt động ổn định và an toàn.
Khối hiển thị và thông báo	- Cung cấp phản hồi trực quan và cảnh báo đến người dùng. - Thiết bị này có thể là LCD, Blynk

Bảng 3.3.1: Mô tả chức năng các khối

3.4 Thiết kế phần cứng

Căn cứ vào “Sơ đồ kiến trúc hệ thống” và “Đặc tả kỹ thuật” lựa chọn các linh kiện:

STT	Khối chức năng	Linh kiện	Mô tả
1	Khối đầu vào	Bàn phím 4x3	- Cho phép người dùng nhập mật khẩu để mở cửa. - Dữ liệu nhập từ bàn phím sẽ được gửi đến vi điều khiển để xử lý.
2	Khối điều khiển	ESP32 WROOM 32 PINOUT	- Nhận tín hiệu từ các thiết bị đầu vào (bàn phím, cảm biến). - Xử lý logic, điều khiển các thiết bị đầu ra như servo, LCD, buzzer. - Giao tiếp với ứng dụng Blynk để gửi dữ liệu cảm biến và nhận lệnh điều khiển từ xa.
3	Khối hiển thị thông báo	Màn hình LCD	- Hiển thị trạng thái hệ thống, mật khẩu nhập vào. - Cung cấp phản hồi trực quan cho người dùng về nhiệt độ và độ ẩm trong nhà.
4	Khối vận hành	SG90	- Mở hoặc đóng cửa khi được lệnh từ khối điều khiển. - Hoạt động như một cơ cấu chấp hành để thực hiện hành động vật lý.
5	Khối cảm biến	DHT11, PIR	- DHT11: Đo nhiệt độ và độ ẩm trong nhà. - PIR: Phát hiện chuyển động trong khu vực phòng khách. - Gửi dữ liệu về cho vi điều khiển để xử lý và thực hiện yêu cầu từ vi điều khiển.

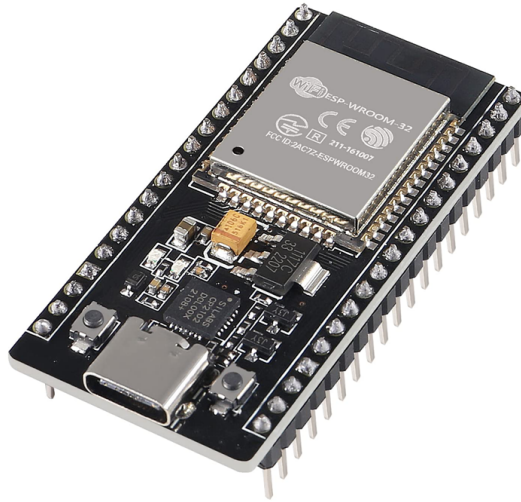
Bảng 3.4.1: Mô tả chức năng các khối

- **Thông số kỹ thuật của linh kiện**

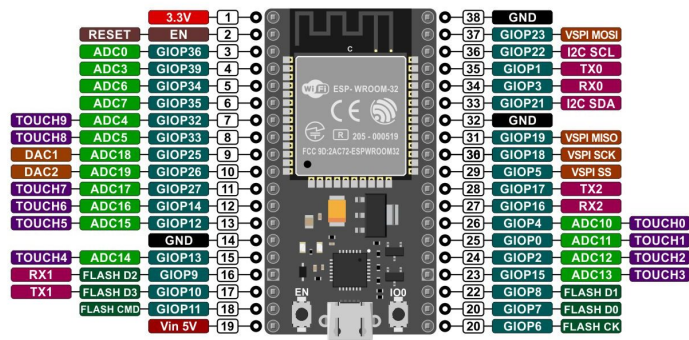
- **ESP32 WROOM 32**

- * Bộ vi xử lý: Bộ vi xử lý Xtensa lõi kép (hoặc lõi đơn) 32-bit LX6, hoạt động ở tần số 240 MHz (160 MHz cho ESP32-S0WD và ESP32-U4WDH) và hoạt động ở tối đa 600 MIPS (200 MIPS với ESP32-S0WD/ESP32-U4WDH).
- * Bộ nhớ gồm có:
 - 448 KB ROM
 - 520 KB SRAM
 - 8 KB SRAM trong RTC.
- * Bộ nhớ Flash: 4MB.

- * Tần số hoạt động: 2.4GHz
- * Điện áp hoạt động/nguồn cấp: 3,3-5V.
- * Dòng tiêu thụ trung bình: 80 mA.
- * Nhiệt độ hoạt động: -40°C đến 85°C.



Hình 3.4.1: ESP32 WROOM 32.

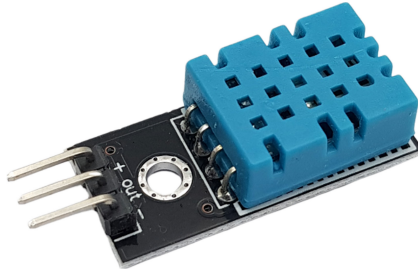


Hình 3.4.2: Hình 3.5.2: Sơ đồ chân ESP32 WROOM 32.

– DHT11

- * Nguồn 3 -> 5V.
- * Dòng sử dụng: 2.5 mA MAX
- * Đo tốt ở độ ẩm 20 đến 70%, sai số 5%
- * Đo tốt ở nhiệt độ 0 đến 50 độ C, sai số 2%
- * Kích thước 15mm x 12mm x 5.5 mm.
- * 4 chân, khoảng cách chân 0.1 inch.
- * Nhiệt độ hoạt động: -40°C đến 85°C.
- * Sơ đồ chân

- VCC : Cấp nguồn 5 VDC.
- GND : Cấp nguồn 0V.
- OUT : Truyền dữ liệu qua giao tiếp 1-wire.



Hình 3.4.3: Cảm biến nhiệt độ, độ ẩm DHT11

– **Cảm biến chuyển động PIR HC - SR501**

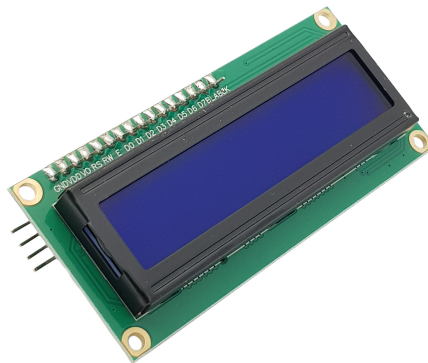
- * Điện áp hoạt động là 5 V.
- * Dòng điện tiêu thụ: 65mA
- * Thời gian trễ: Điều chỉnh (0,3 giây – 18 giây)
- * Phạm vi cảm ứng: góc quét $< 100^\circ$ và xa 7m
- * Nhiệt độ hoạt động: -20°C – $+80^\circ\text{C}$
- * Kích thước mạch: 32 * 24mm, lỗ vít 28mm, đường kính vít 2 mm, nắp cảm ứng (đường kính): 23mm
- * Sơ đồ chân
 - VCC : Cấp nguồn 5 VDC.
 - GND : Cấp nguồn 0V.
 - OUT : Xuất tín hiệu số mức cao (3.3V) khi phát hiện chuyển động, mức thấp (0V) khi không có chuyển động.



Hình 3.4.4: Cảm biến PIR HC - SR501

– **Màn hình hiển thị LCD**

- * Điện áp hoạt động là 5 V.
- * Dòng điện tiêu thụ: 350uA - 600uA
- * Nhiệt độ hoạt động: -30°C đến 75°C.
- * Kích thước 96 x 60 mm, chữ trắng, nền xanh dương
- * Đèn Led nền có thể điều khiển bằng biến trở hoặc PWM
- * Địa chỉ I2C: 0x27.
- * Trọng lượng 38g.
- * Giao tiếp: I2C.
- * Sơ đồ chân
 - VCC : Cấp nguồn 5.5 VDC.
 - GND : Cấp nguồn 0V.
 - SCL : Cấp xung Clock cho chuẩn I2C.
 - SDA : Cấp xung Data cho chuẩn I2C



Hình 3.4.5: Màn hình LCD 16x02

– **Động cơ servo SG90**

- * Điện áp hoạt động là 5 V.
- * Dòng điện tiêu thụ: 65mA
- * Tốc độ: 0.12 sec/ 60 deg (4.8VDC)
- * Lực kéo: 1.6 Kg.cm
- * Góc quay: 180 độ
- * Kích thước: 21x12x22mm
- * Trọng lượng: 9g.
- * Sơ đồ chân
 - VCC (đỏ) : Cấp nguồn 5 VDC.
 - GND (đen): Cấp nguồn 0V.
 - OUT (cam): Nhận xung PWM để điều khiển góc quay servo.



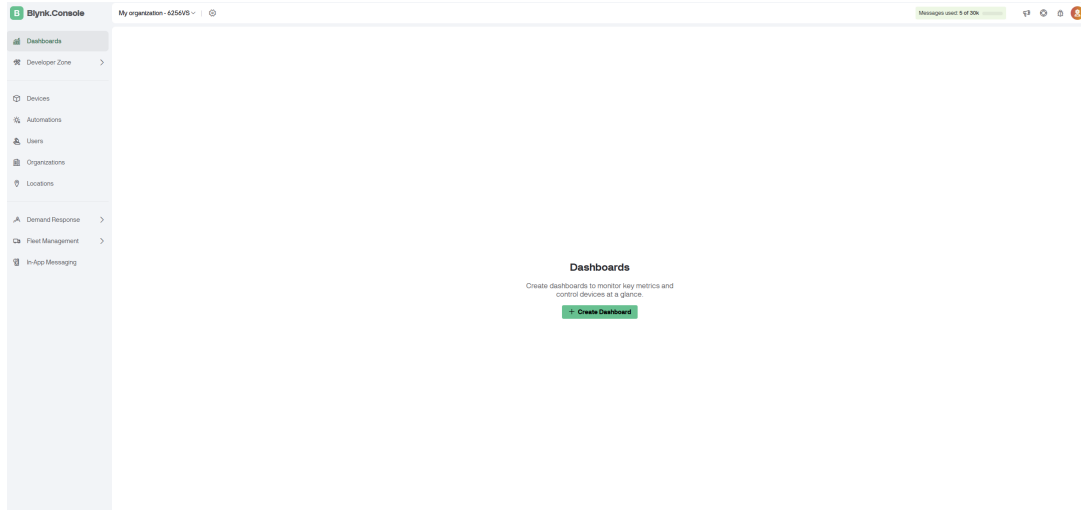
Hình 3.4.6: Động cơ servo SG90

– **Các linh kiện khác: LED, BUZZER**

=> Giúp hệ thống phản hồi nhanh chóng bằng ánh sáng và âm thanh khi người dùng yêu cầu đồng thời cảnh báo khi gặp nguy hiểm, từ đó nâng cao trải nghiệm và độ an toàn cho người dùng.

3.5 Thiết kế phần mềm

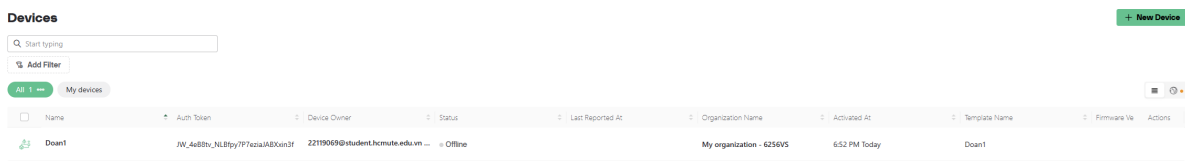
3.5.1 Thiết lập Blynk



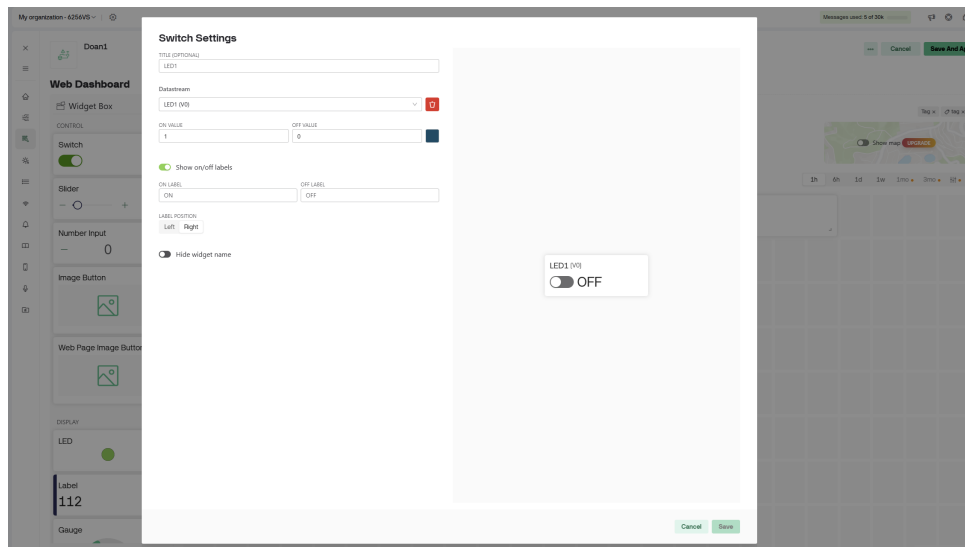
Hình 3.5.1: Màn hình quản lý của Blynk

Sau khi đăng nhập vào web Blynk 2.0 trên máy tính, giao diện chính hiển thị các mục như Devices, Templates, Datastreams và Automations. Đây là nơi người dùng sẽ theo dõi và cấu hình toàn bộ hệ thống. Để bắt đầu, người dùng tiến hành tạo một Template mới, đặt tên cho template, chọn loại board phù hợp là ESP32, và chọn phương thức kết nối là WiFi. Trong template này, chọn thiết lập các Datastreams, là Virtual Pin V0, V1,... để ánh xạ dữ liệu giữa thiết bị phần cứng và giao diện điều khiển trên ứng dụng.

Sau khi hoàn tất việc cấu hình template, bước tiếp theo là tạo một thiết bị mới (Device) dựa trên template vừa tạo. Ứng dụng sẽ tự động sinh ra một Auth Token, đây là mã định danh duy nhất dùng để lập trình cho vi điều khiển ESP kết nối với nền tảng Blynk Cloud. Token này cần được khai báo trong đoạn mã chương trình của ESP để thiết bị có thể truyền và nhận dữ liệu với server.

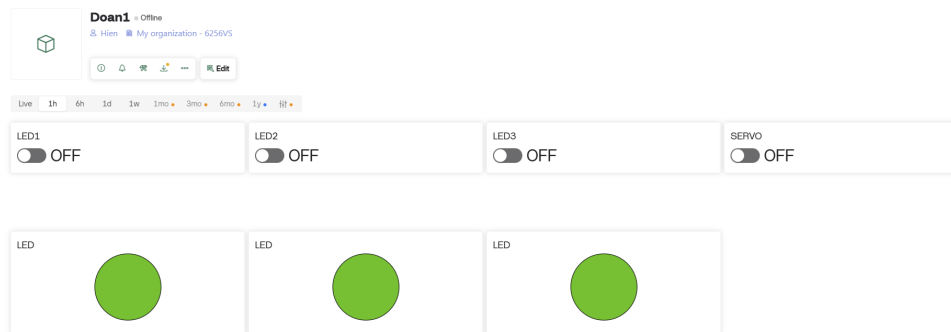


Hình 3.5.2: Tạo devices cho Blynk



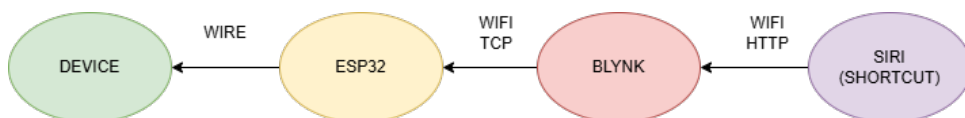
Hình 3.5.3: Tạo công tắc ảo

Khi thiết bị đã được tạo thành công, người dùng tiến hành thiết kế giao diện điều khiển bằng cách thêm các widget như nút nhấn, thanh trượt, hoặc hiển thị giá trị. Mỗi widget sẽ được gán với một Virtual Pin tương ứng đã khai báo trong phần Datastreams. Giao diện này chính là nơi người dùng tương tác với hệ thống như bật/tắt đèn, mở khóa cửa, đọc trạng thái cảm biến,... Các dữ liệu sẽ được truyền giữa thiết bị và ứng dụng theo thời gian thực thông qua Blynk Cloud.



Hình 3.5.4: Màn hình Blynk sau khi tạo các công tắc ảo

3.5.2 Thiết lập Siri và Blynk



Hình 3.5.5: Dữ liệu từ Siri đến Devices

Hệ thống điều khiển thiết bị bằng giọng nói sử dụng trợ lý ảo Siri, nền tảng điều khiển Blynk và vi điều khiển ESP32 để thực hiện các tác vụ điều khiển thiết bị điện một cách linh hoạt và gần như tức thời. Khi người dùng ra lệnh bằng giọng nói thông qua Siri trên iPhone, Siri sẽ kích hoạt một Shortcut được lập trình sẵn. Bên trong Shortcut này có chứa một yêu cầu HTTP với đầy đủ thông tin như địa chỉ URL, phương thức gửi (GET), và các tham số truyền tải dữ liệu điều khiển.

Cụ thể, URL được sử dụng có dạng: `http://blynk.cloud/external/api/update?token=KsYqu00-45jfyXAWtUTUEwyircm_mcd&pin=V3&value=0`, trong đó `token` là mã xác thực tài khoản của người dùng trên nền tảng Blynk, `pin=V3` là chân điều khiển ảo tương ứng với thiết bị cần tác động (trong trường hợp này là đèn), và `value=0` là giá trị điều khiển (0 để tắt, 1 để bật). Ngay khi Shortcut được gọi, yêu cầu HTTP này sẽ được gửi trực tiếp đến máy chủ Blynk thông qua kết nối WiFi.

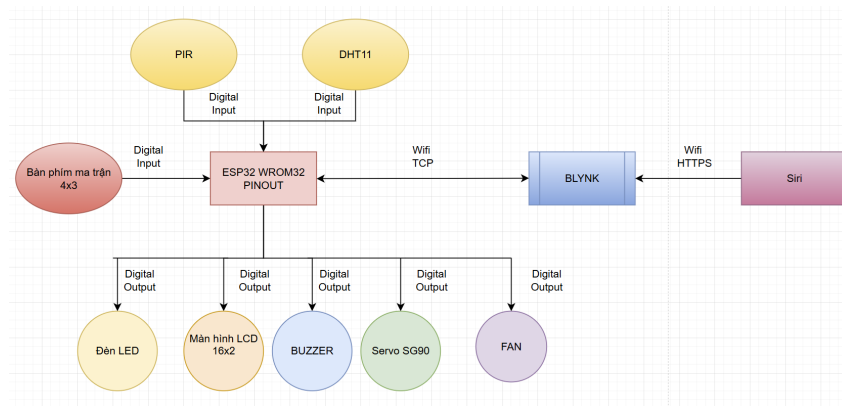
Blynk server, sau khi nhận được yêu cầu, sẽ tiến hành xử lý thông qua hệ thống API nội bộ. Dựa vào `token` và thông tin chân điều khiển, hệ thống xác định đúng thiết bị được liên kết và truyền tiếp lệnh điều khiển đến vi điều khiển ESP32. Quá trình này sử dụng giao thức TCP thông qua kết nối WiFi, đảm bảo độ tin cậy cao và hạn chế mất gói dữ liệu. ESP32 sau đó nhận tín hiệu, giải mã dữ liệu theo chuẩn của Blynk protocol, và thực hiện chuyển đổi tín hiệu này thành hành động điều khiển thực tế.

Thiết bị đầu ra (trong trường hợp này là đèn) được nối trực tiếp vào các chân GPIO của ESP32 thông qua dây dẫn vật lý (WIRE). Điều này đảm bảo tính ổn định trong kết nối và giúp thiết bị phản hồi nhanh chóng với độ trễ rất thấp. Nhờ cơ chế này, người dùng có thể ra lệnh bằng giọng nói để bật/tắt các thiết bị điện trong nhà một cách dễ dàng, không cần mở ứng dụng hay thao tác trực tiếp.

Chương 4

THI CÔNG VÀ ĐÁNH GIÁ

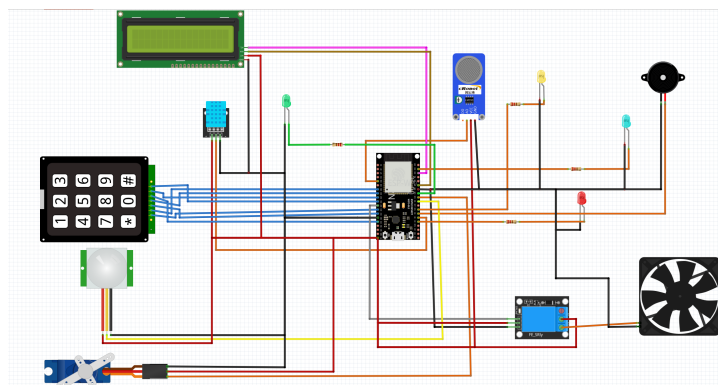
4.1 Thi công



Hình 4.1.1: Sơ đồ kết nối các module phần cứng và phần mềm.

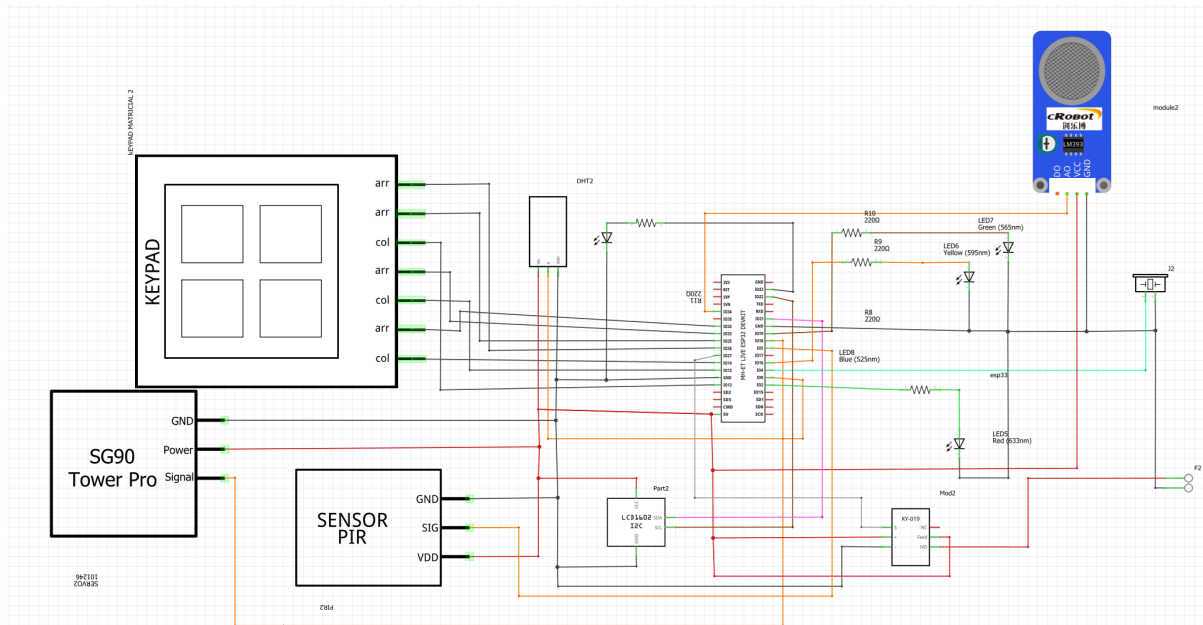
4.1.1 Thi công phần cứng

- Sơ đồ kết nối



Hình 4.1.2: Sơ đồ kết nối.

- Sơ đồ nguyên lí



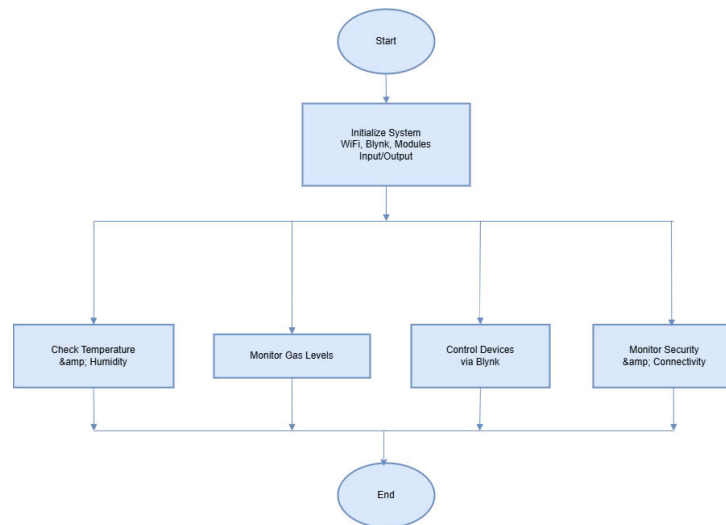
Hình 4.1.3: Sơ đồ nguyên lí.

4.1.2 Nguyên lí hoạt động

- Sau khi khởi động, ESP32 tiến hành kết nối với mạng WiFi được cấu hình sẵn. Người dùng có thể nhấn nút * trên bàn phím 4x3 để bắt đầu quá trình nhập mật khẩu. Khi nhập đúng, hệ thống chuyển sang trạng thái mở cửa, đồng thời hiển thị thông báo trên LCD, điều khiển servo quay 90 độ để mở cửa, sau đó trở lại vị trí ban đầu. Nếu nhập sai, người dùng phải đợi 10 giây thì mới nhập lại được.
- Tiếp đó, ở phòng khách, hệ thống sẽ đọc cảm biến DHT11 để lấy nhiệt độ và độ ẩm môi trường. Giá trị đọc được sẽ được hiển thị trên màn hình LCD 16x2 và đồng thời cũng hiển thị lên Blynk. Ngoài ra, còn có một đèn LED sẽ được bật sáng nếu PIR phát hiện chuyển động. Hơn thế nữa, LED này còn có thể được bật tắt thủ công thông qua ứng dụng Blynk.
- Ở phòng bếp, cảm biến khí gas analog liên tục đo nồng độ khí trong bếp mỗi 2 giây. Khi giá trị vượt ngưỡng cho phép, hệ thống sẽ kích hoạt còi và đèn cảnh báo, đồng thời gửi thông báo “Nguy hiểm!” lên Blynk. Nếu khí trở lại mức an toàn, còi sẽ tắt và hiển thị trạng thái “An toàn”. Tại đây, người dùng cũng có thể điều khiển đèn thông qua Blynk.
- Ở phòng ngủ, người dùng điều khiển đèn LED và quạt thông qua Blynk. Ngoài ra, đèn và quạt của toàn bộ hệ thống sẽ được điều khiển thông qua giọng nói trên điện thoại iPhone bằng Siri.

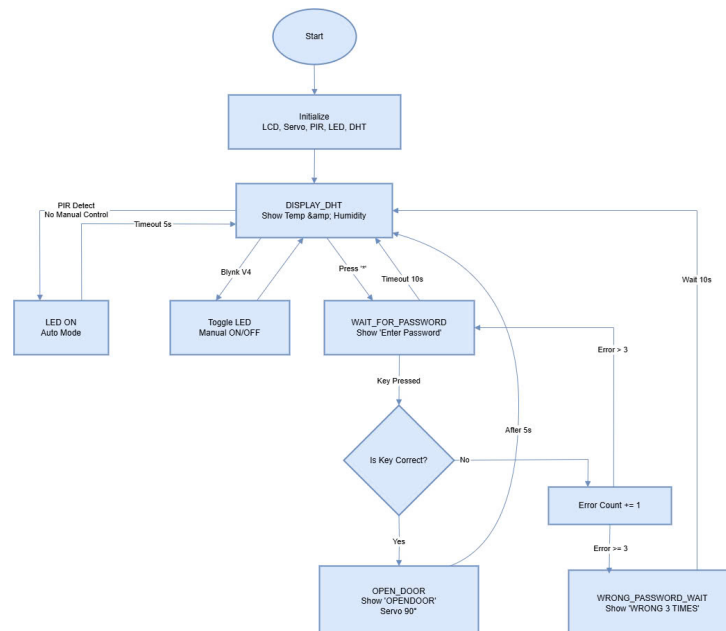
4.1.3 Lưu đồ giải thuật

- Lưu đồ tổng quát



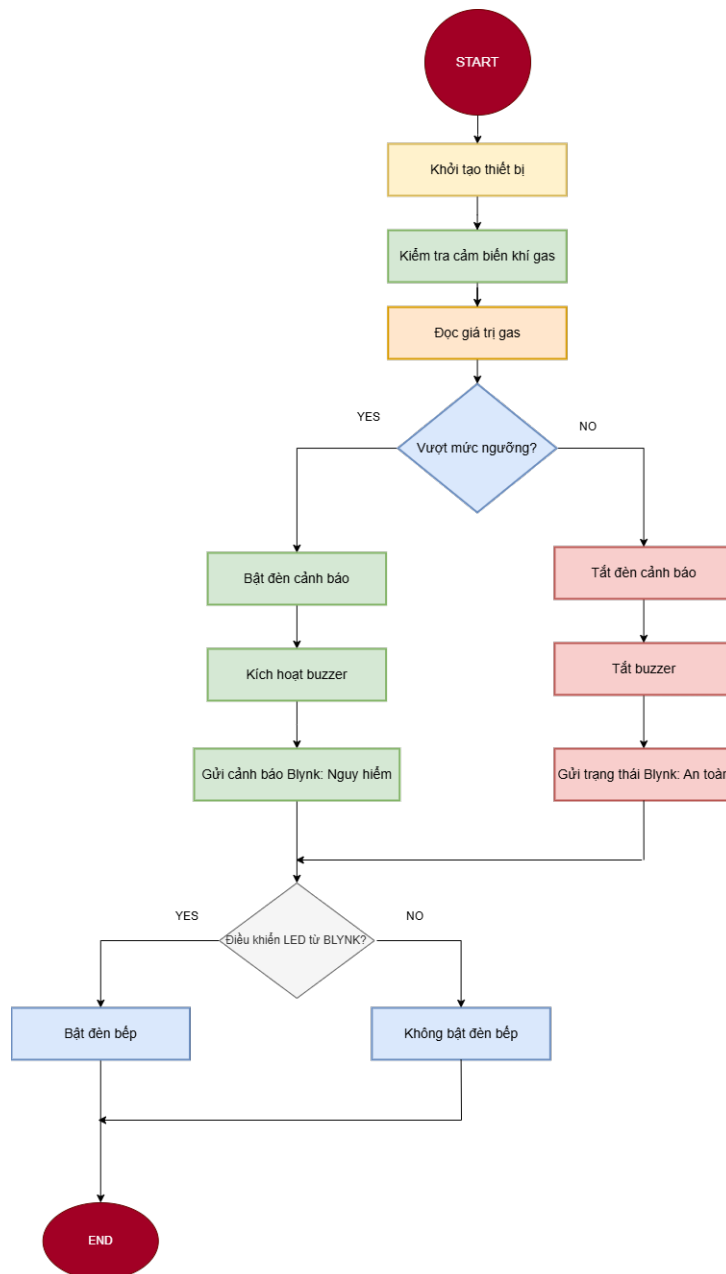
Hình 4.1.4: Lưu đồ tổng quát của hệ thống nhà thông minh.

- Lưu đồ phòng khách



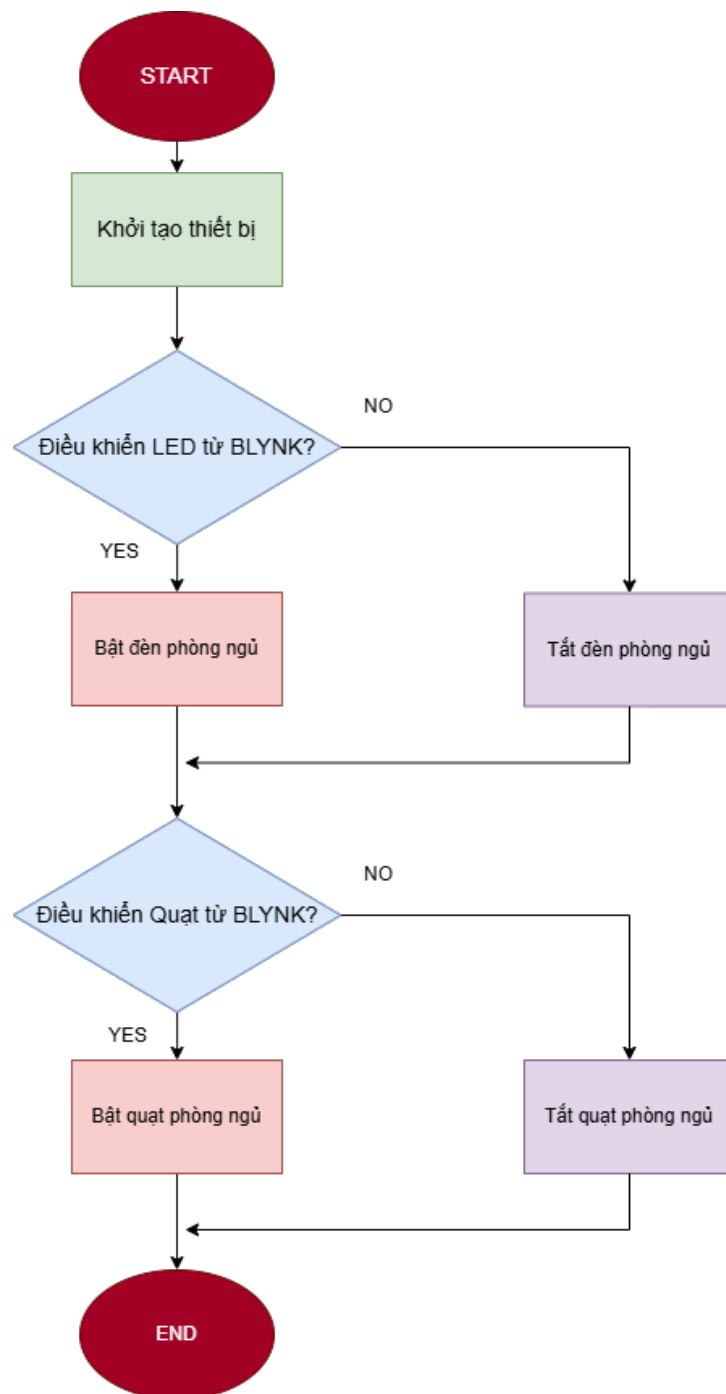
Hình 4.1.5: Lưu đồ phòng khách của hệ thống nhà thông minh.

- Lưu đồ phòng bếp



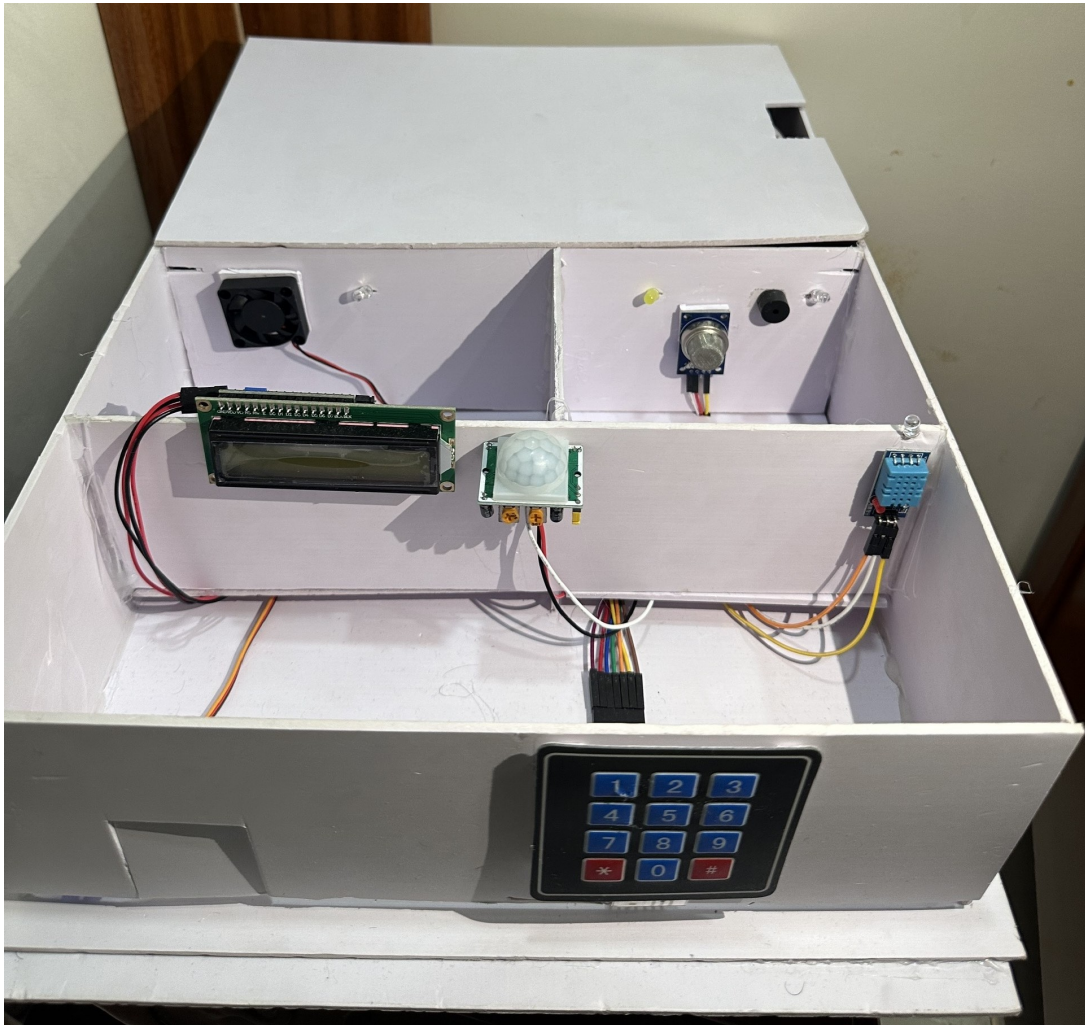
Hình 4.1.6: Lưu đồ phòng bếp của hệ thống nhà thông minh.

- Lưu đồ phòng ngủ



Hình 4.1.7: Lưu đồ phòng ngủ của hệ thống nhà thông minh.

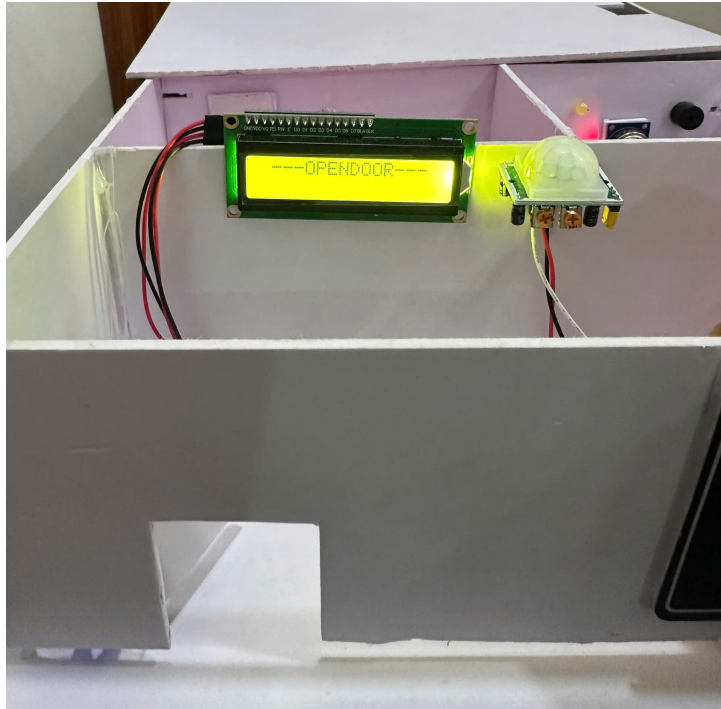
4.2 Kết quả



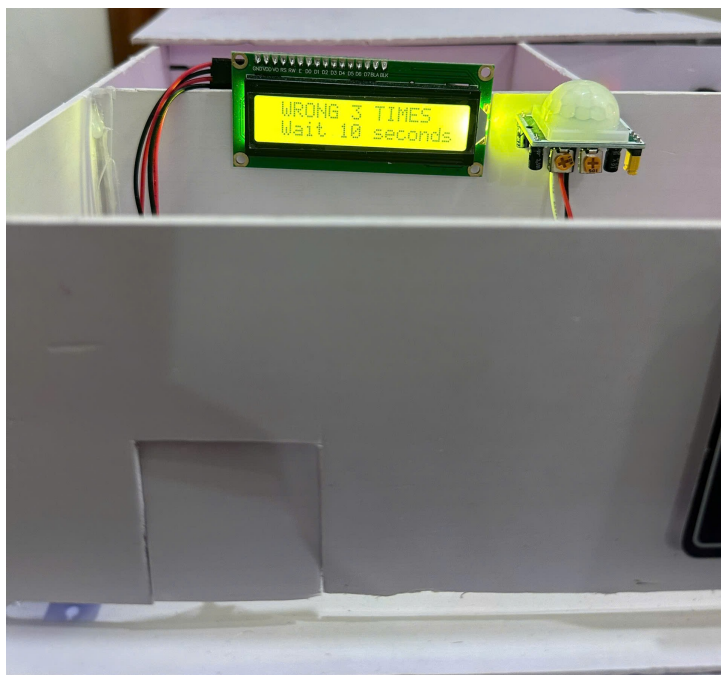
Hình 4.2.1: Mô hình sản phẩm

Khi người dùng sử dụng phòng khách:

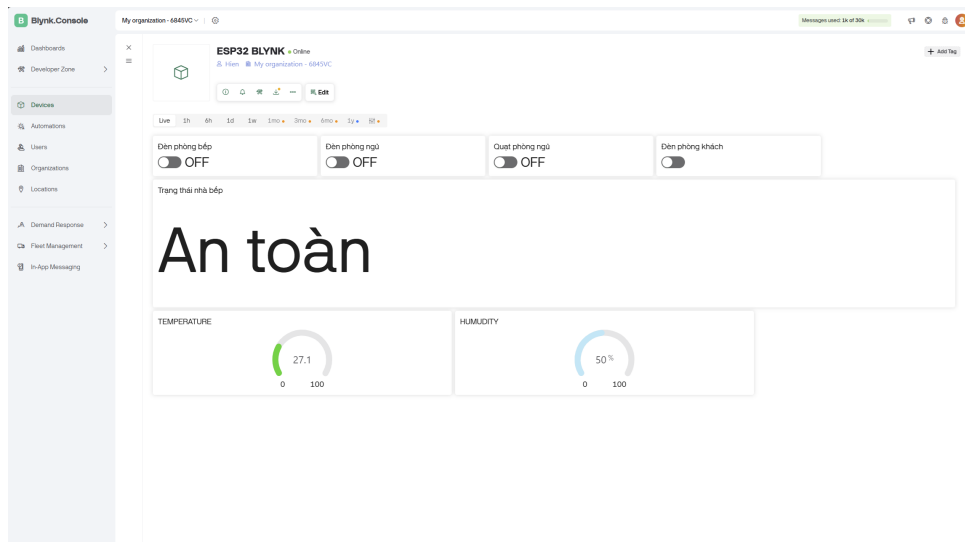
- Người dùng mở cửa thành công với mật khẩu đúng.
- Khi người dùng mật sai mật khẩu, sẽ được thử lại sau 10s.
- Màn hình LCD sẽ hiển thị thông số nhiệt độ, độ ẩm cùng với trạng thái đèn phòng khách (Auto/Manual).



Hình 4.2.2: Cửa mở khi mật khẩu đúng.



Hình 4.2.3: Cửa mở khi mật khẩu đúng.



Hình 4.2.4: Giao diện blynk hiển thị nhiệt độ, độ ẩm.

Khi hệ thống cảnh báo rò rỉ khí gas

- Đèn LED và BUZZER sẽ liên tục sáng và kêu tới khi lượng khí gas giảm về mức an toàn.
- Trên Blynk, sẽ hiện chữ cảnh báo nguy hiểm.



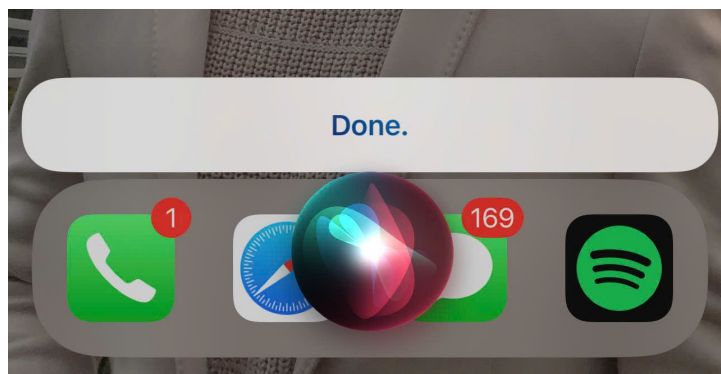
Hình 4.2.5: Giao diện Blynk thông báo nguy hiểm.



Hình 4.2.6: Đèn và còi sẽ hú khi nguy hiểm,

Khi người dùng sử dụng giọng nói để điều khiển thiết bị

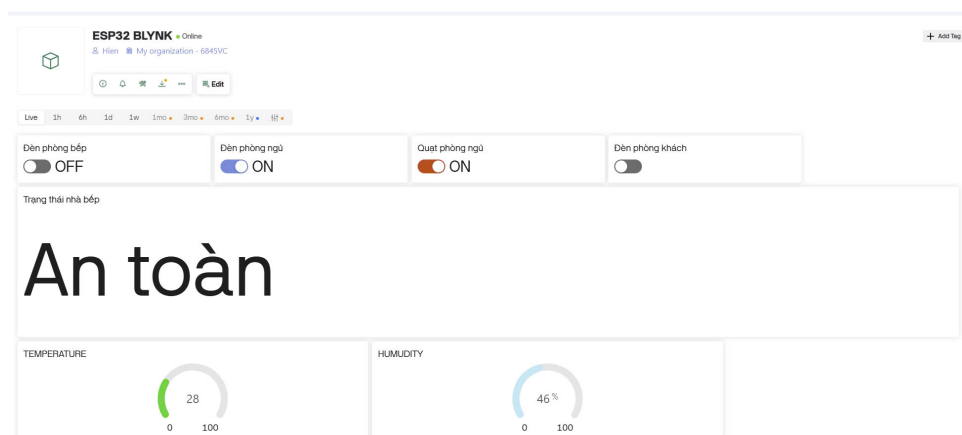
- Sau khi người dùng sử dụng giọng nói để điều khiển thiết bị
- Trên điện thoại Siri sẽ thông báo thành công từ đó trên giao diện Blynk, các công tắc ảo sẽ được bật.



Hình 4.2.7: Sử dụng Siri thành công



Hình 4.2.8: Đèn đã sáng và quạt đã xoay.



Hình 4.2.9: Giao diện blynk mà người dùng sử dụng Siri bật đèn và quạt

4.3 Đánh giá hệ thống

Sau một thời gian triển khai và vận hành hệ thống nhà thông minh trong môi trường thực tế, nhóm đã tiến hành đánh giá hiệu quả hoạt động và những thách thức còn tồn tại để từ đó đề xuất các hướng tối ưu trong tương lai.

Ưu điểm:

Hệ thống hoạt động ổn định và đáp ứng tốt các yêu cầu đã đề ra. Các chức năng điều khiển thiết bị qua giọng nói, điều khiển từ xa qua ứng dụng Blynk, kiểm soát an toàn nhà bếp (phát hiện khí gas), giám sát nhiệt độ/độ ẩm và điều khiển thiết bị bằng bàn phím vật lý đều vận hành trơn tru. Hệ thống đã được thử nghiệm trong nhiều trường hợp sử dụng và điều kiện vận hành khác nhau, cho thấy tính thích nghi cao.

Độ trễ trong phản hồi là khá thấp, đảm bảo trải nghiệm mượt mà và tức thời cho người dùng. Việc sử dụng ESP32 kết hợp với hệ điều hành thời gian thực FreeRTOS giúp phân chia và quản lý tác vụ hiệu quả, đồng thời hạn chế xung đột giữa các chức năng đang hoạt động song song.

Ngoài ra, việc tích hợp điều khiển giọng nói qua Siri Shortcuts tạo sự tiện lợi, giúp người dùng dễ dàng điều khiển các thiết bị trong sinh hoạt hàng ngày mà không cần tiếp xúc vật lý, đặc biệt hữu ích trong các tình huống tay bận hoặc người khuyết tật.

Hạn chế:

Một trong những hạn chế lớn nhất của hệ thống là phụ thuộc vào kết nối mạng Wi-Fi. Khi mất kết nối Internet, các chức năng điều khiển từ xa qua ứng dụng Blynk sẽ không thể sử dụng, đồng thời các tính năng chuyển chế độ hoạt động thông qua cảm biến PIR cũng bị ảnh hưởng nếu phụ thuộc vào dữ liệu từ server. Điều này làm giảm đáng kể khả năng tự động hóa và điều khiển thông minh trong thời gian mất kết nối.

Ngoài ra, mặc dù ESP32 hỗ trợ nhiều tính năng mạnh mẽ, nhưng khi phải xử lý đồng thời nhiều cảm biến cùng với các tác vụ điều khiển và giao tiếp mạng, có thể xuất hiện tình trạng quá tải tài nguyên, dẫn đến hiện tượng treo nhẹ hoặc phản hồi chậm nếu không được tối ưu hợp lý.

Hệ thống hiện tại cũng chưa được trang bị các cơ chế bảo mật nâng cao như mã hóa dữ liệu truyền tải hay xác thực người dùng, điều này có thể gây rủi ro trong các môi trường nhiều người dùng hoặc khi kết nối từ xa qua Internet.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Hệ thống nhà thông minh sử dụng ESP32 kết hợp với các thiết bị cảm biến như DHT11, PIR, keypad, LCD, servo và tích hợp nền tảng Blynk đã góp phần tự động hóa các thao tác điều khiển thiết bị trong gia đình, từ đó mang lại sự tiện lợi và nâng cao trải nghiệm người dùng. Việc tích hợp điều khiển bằng giọng nói thông qua Siri giúp người dùng có thể tương tác với hệ thống một cách linh hoạt và hiện đại hơn, phù hợp với xu hướng phát triển của các hệ thống nhà thông minh hiện nay. Thông qua cơ chế quản lý thiết bị theo phòng (phòng bếp, phòng ngủ), hệ thống cho phép điều khiển theo ngữ cảnh cụ thể, kết hợp với khả năng giám sát điều kiện môi trường (nhiệt độ, chuyển động) và tự động phản hồi, tạo nên một không gian sống thông minh và an toàn.

Trên thực tế, hệ thống đã hoạt động ổn định, đáp ứng được các yêu cầu điều khiển cơ bản. Tuy nhiên, vẫn còn tồn tại một số hạn chế thực tế như:

- Phụ thuộc vào kết nối mạng: Khi không có Internet hoặc kết nối mạng không ổn định, các tính năng điều khiển từ xa thông qua Blynk và Siri sẽ bị gián đoạn, gây ảnh hưởng đến trải nghiệm người dùng.
- Độ trễ khi điều khiển qua Siri: Mặc dù chức năng nhận lệnh giọng nói đã vận hành tốt, nhưng vẫn còn một độ trễ nhỏ trong quá trình thực thi lệnh, đặc biệt khi phải kết nối đến máy chủ trung gian hoặc Blynk server.
- Khả năng bảo mật còn hạn chế: Một số chức năng quan trọng vẫn đang sử dụng cơ chế xác thực đơn giản như nhập mật khẩu, chưa thật sự an toàn nếu hệ thống triển khai ở môi trường nhiều người dùng.
- Thiết bị phần cứng hạn chế khả năng mở rộng: Với ESP32, việc xử lý các tác vụ phức tạp hơn như nâng cấp khóa bằng nhận diện khuôn mặt trong tương lai,.. bị giới hạn. Đây là yếu tố cần xem xét khi phát triển hệ thống trong tương lai.

5.2 Hướng phát triển

Để xây dựng một hệ thống nhà thông minh tối ưu và bền vững trong tương lai, việc tiếp tục nâng cấp về phần cứng và phần mềm là cần thiết. Một hướng phát triển quan trọng là thay thế vi điều khiển ESP32 bằng Raspberry Pi, nhằm tận dụng hiệu năng xử lý mạnh mẽ hơn, khả năng hỗ trợ đa luồng tốt hơn và tính linh hoạt cao khi triển khai các thuật toán phức tạp khác.

Song song với việc nâng cấp phần cứng, hệ thống cũng có thể được cải tiến bằng cách thay thế cơ chế nhập mật khẩu truyền thống bằng công nghệ nhận diện khuôn mặt. Việc này không chỉ tăng cường bảo mật mà còn nâng cao tính tiện lợi cho người dùng, đặc biệt trong các tình huống cần thao tác nhanh hoặc người dùng không thể nhập liệu thủ công. Ngoài ra, hệ thống nhận diện khuôn mặt có thể được phát triển thêm để nhận biết các thành viên trong gia đình và tự động kích hoạt các ngữ cảnh điều khiển thiết bị tùy theo người sử dụng. Ví dụ, khi trẻ em được nhận diện, hệ thống sẽ hạn chế quyền điều khiển các thiết bị nguy hiểm hoặc truy cập các chức năng nâng cao.

Một hướng mở rộng khác là tăng cường khả năng kết nối và tích hợp với các nền tảng điều khiển nhà thông minh hiện có như Apple HomeKit, Google Home, hoặc Amazon Alexa. Điều này sẽ giúp hệ thống hoạt động mượt mà trong hệ sinh thái đa thiết bị, đồng thời khai thác hiệu quả hơn các tính năng điều khiển bằng giọng nói và tự động hóa kịch bản.

Cuối cùng, giao diện người dùng cũng cần được thiết kế lại theo hướng hiện đại, trực quan và thích ứng tốt với nhiều loại thiết bị như điện thoại, máy tính bảng, hoặc màn hình cảm ứng treo tường. Khả năng tùy biến linh hoạt theo người dùng và ngữ cảnh sử dụng sẽ là yếu tố then chốt giúp hệ thống trở nên thân thiện và dễ triển khai trong thực tế.

TÀI LIỆU THAM KHẢO

- E. Mahesh Babu, K. Satheesh, B. Mohith, J. Madhu Sudhan. *IoT Based Home Automation Using ESP32 & Google Home*. Materials Science and Technology, 2024. <https://materialsciencetech.com/mst/uploads/2024-42461.pdf>
- Thegioiic. *ESP32 - Giới thiệu module, Thông số kỹ thuật*. 08 September 2022. [Trực tuyến]. Truy cập ngày 21 October 2024. <https://www.thegioiic.com/tin-tuc/esp32-gioi-thieu-module-thong-so-ky-thuat>
- DienthongminhEsmart. *Cách sử dụng Blynk IOT với ESP32*. 02 November 2024. [Trực tuyến]. Truy cập ngày 19 November 2024. <https://dienthongminhesmart.com/lap-trinh-esp32/blynk-iot-vaesp32>
- Deniz Arsan, Ali Zaidi, Aravind Sagar, Ranjitha Kumar. *App-Based Task Shortcuts for Virtual Assistants*. In Proceedings of the 2021 5th International Conference on Conversational User Interfaces (CUI '21), ACM, 2021. <https://dl.acm.org/doi/abs/10.1145/3472749.3474808>
- Dejan Babić, Ivan Jovović, Tomo Popović, Nataša Kovač, Stevan Čakić. *An Internet of Things System for Environmental Monitoring Based on ESP32 and Blynk*. In 2022 29th International Conference on Systems, Signals and Image Processing (IWSSIP), IEEE, 2022. <https://ieeexplore.ieee.org/abstract/document/9743538>